# DATABRICKS

## Data Engineer Associate

Certification Exam Prep Guide

All 7 Exam Sections • Key Concepts • Commands & Code • Quick-Recall Tables

| Section | Weight | Priority |
|---|---|---|
| 1. Databricks Intelligence Platform | **6%** | Medium |
| 2. Data Ingestion and Loading | **21%** | **HIGH** |
| 3. Data Transformation and Modeling | **22%** | **HIGH** |
| 4. Working with Lakeflow Jobs | **16%** | **HIGH** |
| 5. Implementing CI/CD | **10%** | Medium |
| 6. Troubleshooting & Optimization | **10%** | Medium |
| 7. Governance and Security | **15%** | **HIGH** |

# Section 1: Databricks Intelligence Platform (6%)

## 1.1 Core Architecture

The Databricks platform is built on a lakehouse architecture combining data lake storage with data warehouse capabilities.

### Key Architecture Layers

- Cloud Infrastructure Layer — runs on AWS, Azure, or GCP
- Delta Lake — open-source storage layer with ACID transactions
- Unity Catalog — unified governance and security layer
- Compute Layer — clusters, SQL Warehouses, serverless
- Workspace — collaboration interface for notebooks, jobs, and assets

## 1.2 Delta Lake

Delta Lake is the foundational storage format. Know these core capabilities:

| Feature | Description |
|---|---|
| **ACID Transactions** | Ensures data integrity across concurrent reads and writes |
| **Time Travel** | Query previous table versions using VERSION AS OF or TIMESTAMP AS OF |
| **Schema Enforcement** | Rejects data that doesn't match the table schema on write |
| **Schema Evolution** | Allows adding new columns via MERGE Schema or ALTER TABLE |
| **Unified Batch & Streaming** | Same API for both batch and streaming workloads |
| **Z-Ordering** | Multi-dimensional clustering for faster query pruning |
| **Liquid Clustering** | Replaces Z-Order; auto-adjusts cluster keys as data grows |

## 1.3 Unity Catalog

Unity Catalog is the centralized governance solution. Hierarchy you must know:

- Metastore (top-level, one per region)
- **Catalog**
  - Schema (Database)
  - Table / View / Volume / Function

## 1.4 Compute Services — Quick Comparison

| Compute Type | Best For | Autoscaling | Cost Model |
|---|---|---|---|

| | | | |
|---|---|---|---|
| All-Purpose Cluster | Development & ad-hoc work | Yes | DBU per hour + VM |
| Job Cluster | Automated pipeline runs | Yes | DBU per hour + VM (cheaper) |
| SQL Warehouse | BI & SQL analytics | Yes (serverless) | DBU per query |
| Serverless Compute | Fast-start notebooks & jobs | Auto | Per second, no idle cost |

> ⚠ **Exam Tip:** Job Clusters are terminated after the job completes, making them more cost-effective than All-Purpose clusters for production pipelines.

# Section 2: Data Ingestion and Loading (21%)

## 2.1 Ingestion Patterns Overview

| Pattern | Method | Frequency | Use Case |
|---|---|---|---|
| Batch | COPY INTO / Auto Loader | Scheduled | Large file loads |
| Incremental | Auto Loader checkpoint | Continuous | New files only |
| Streaming | Auto Loader streaming | Real-time | Event / sensor data |
| CDC | Lakeflow Connect | Near real-time | DB change capture |

## 2.2 COPY INTO Command

Use COPY INTO for idempotent, incremental loading from cloud object storage into Delta tables.

```
COPY INTO my_catalog.my_schema.target_table
FROM 'abfss://container@account.dfs.core.windows.net/path/'
FILEFORMAT = PARQUET
FILES = ('file1.parquet', 'file2.parquet')  -- optional
COPY_OPTIONS ('mergeSchema' = 'true')
;
```

- Tracks loaded files using a metadata table — safe to re-run (idempotent)
- Supports CSV, JSON, Parquet, Avro, ORC, Text, Binary
- Use FORCE = TRUE to reload files that were already processed
- Best for cloud storage → Delta table with known/stable schemas

## 2.3 Auto Loader

Auto Loader (cloudFiles) continuously ingests new files from cloud storage with schema detection and evolution.

### Key Configuration Options

| Option | Purpose |
|---|---|
| `cloudFiles.format` | File format: json, csv, parquet, etc. |
| `cloudFiles.schemaLocation` | Path to store inferred schema |
| `cloudFiles.schemaEvolutionMode` | addNewColumns / rescue / failOnNewColumns / none |
| `cloudFiles.inferColumnTypes` | true to infer types instead of string default |
| `cloudFiles.useNotifications` | true = file notification mode (low latency) |
| `cloudFiles.maxFilesPerTrigger` | Limit files per micro-batch |

### Auto Loader Code Pattern

```
df = (spark.readStream
  .format('cloudFiles')
  .option('cloudFiles.format', 'json')
  .option('cloudFiles.schemaLocation', '/mnt/schema_path')
  .option('cloudFiles.schemaEvolutionMode', 'addNewColumns')
  .load('abfss://container@acct.dfs.core.windows.net/raw/')
)
df.writeStream.format('delta').option('checkpointLocation',
'/mnt/chk').table('catalog.schema.table')
```

### Directory Listing vs File Notification Mode

- Directory Listing — polls cloud storage for new files; simpler but has latency
- File Notification — uses cloud event services (SNS/Event Grid); lower latency, event-driven

> ⚠ **Exam Tip:** Schema Evolution Mode: 'addNewColumns' is the exam-favourite setting. It automatically adds new fields to the target table.

## 2.4 Ingestion Method Decision Matrix

| Method | Volume | Frequency | Governance | Best For |
|---|---|---|---|---|
| Auto Loader | Any | Continuous | Unity Catalog | Files in cloud storage |
| COPY INTO | Small–Med | Batch | Unity Catalog | Idempotent file loads |
| Lakeflow Connect (std) | Large | CDC/Batch | Unity Catalog | Enterprise DBs (JDBC) |
| Lakeflow Connect (mgd) | Large | Near RT | Unity Catalog | SaaS apps (Salesforce) |
| JDBC/ODBC Notebook | Small | On-demand | Manual | Quick prototyping |

## 2.5 Semi-Structured & JSON Data

- Use from_json() to parse JSON strings into structs
- Use schema_of_json() to infer schema from a sample JSON
- Use explode() to convert arrays to rows
- Use col('struct.field') notation to access nested fields

```
from pyspark.sql.functions import from_json, col, schema_of_json, explode
schema = schema_of_json(sample_json_string)
df = df.withColumn('parsed', from_json(col('raw_json'), schema))
df = df.withColumn('item', explode(col('parsed.items')))
```

# Section 3: Data Transformation and Modeling (22%)

## 3.1 Medallion Architecture (Bronze → Silver → Gold)

| Layer | Characteristics | Operations |
|---|---|---|
| Bronze | Raw ingested data as-is; no transformations | Ingest only; append or upsert from source |
| Silver | Cleansed, validated, conformed data | Null checks, dedup, type casting, joins |
| Gold | Aggregated, business-ready data | Aggregations, KPIs, materialized views |

## 3.2 Data Cleaning — Bronze to Silver

### Common Cleaning Operations

- Drop nulls: df.dropna(subset=['col1','col2'])
- Fill nulls: df.fillna({'col1': 0, 'col2': 'unknown'})
- Cast types: df.withColumn('date', col('date_str').cast('date'))
- Filter invalid rows: df.filter(col('amount') > 0)
- Trim whitespace: df.withColumn('name', trim(col('name')))
- Standardize case: df.withColumn('status', upper(col('status')))

```
from pyspark.sql.functions import col, trim, upper, when, regexp_replace
silver_df = (bronze_df
  .dropna(subset=['id', 'event_date'])
  .withColumn('amount', col('amount').cast('double'))
  .withColumn('status', upper(trim(col('status'))))
  .filter(col('amount') > 0)
  .dropDuplicates(['id', 'event_date'])
)
silver_df.write.format('delta').mode('overwrite').saveAsTable('catalog.schema.silver_table')
```

## 3.3 DataFrame Join Operations

| Join Type | Behaviour | Syntax Example |
|---|---|---|
| Inner Join | Only matching rows from both sides | df1.join(df2, 'id', 'inner') |
| Left Join | All rows from left + matching from right | df1.join(df2, 'id', 'left') |
| Broadcast Join | Broadcasts small table; avoids shuffle | df1.join(broadcast(df2), 'id') |
| Cross Join | Cartesian product of both DataFrames | df1.crossJoin(df2) |

| | | |
|---|---|---|
| Union | Stack rows (same schema required) | df1.union(df2) |
| Union All | Same as union (PySpark default) | df1.unionAll(df2) |
| Multiple Keys | Join on multiple columns | df1.join(df2, ['id','date']) |

⚠ **Exam Tip:** Broadcast join threshold is controlled by spark.sql.autoBroadcastJoinThreshold (default 10MB). Set to -1 to disable broadcast joins.

## 3.4 Column & Row Manipulation

### Column Operations

- Add column: df.withColumn('new_col', expr)
- Drop column: df.drop('col_name')
- Rename column: df.withColumnRenamed('old', 'new')
- Split string: split(col('name'), ',')[0]
- Explode array: df.withColumn('item', explode(col('items_array')))

### Aggregation Functions

```
from pyspark.sql.functions import count, countDistinct, approx_count_distinct, mean, sum, avg
df.groupBy('category').agg(
  count('*').alias('total_rows'),
  approx_count_distinct('user_id').alias('est_unique_users'),
  mean('amount').alias('avg_amount'),
  sum('revenue').alias('total_revenue')
).show()
```

💡 **Note:** Use approx_count_distinct() instead of countDistinct() for very large datasets — it's much faster with only ~2% error.

## 3.5 Deduplication

- dropDuplicates() — remove exact duplicate rows
- dropDuplicates(['col1','col2']) — deduplicate by specific columns only
- Use Window functions with row_number() for 'keep latest' deduplication

```
from pyspark.sql.functions import row_number
from pyspark.sql import Window
w = Window.partitionBy('id').orderBy(col('updated_at').desc())
df_deduped = df.withColumn('rn', row_number().over(w)).filter(col('rn') == 1).drop('rn')
```

## 3.6 Spark Tuning Parameters

| Parameter | Default | When to Change |
|---|---|---|

| | | |
|---|---|---|
| spark.sql.shuffle.partitions | 200 | Reduce for small data; increase for large shuffles |
| spark.default.parallelism | cores × 2 | Match to cluster core count for optimal parallelism |
| spark.executor.memory | 1g | Increase when OOM errors occur during processing |
| spark.driver.memory | 1g | Increase for large collect() / broadcast operations |
| spark.sql.autoBroadcastJoinThreshold | 10MB | Lower to force sort-merge joins; -1 to disable |

## 3.7 Gold Layer Object Types

| Object Type | Characteristics | Best Use |
|---|---|---|
| Delta Table | Persisted storage; ACID; versioned | Full historical datasets, CRUD operations |
| View | SQL alias over a query; no storage | Simple abstraction, security layers |
| Materialized View | Pre-computed, auto-refreshed by DLT | BI dashboards requiring fast aggregations |
| Streaming Table | Continuously updated via DLT pipeline | Near real-time metric tracking |

## 3.8 Data Quality Checks

- NOT NULL constraints: ALTER TABLE t ALTER COLUMN id SET NOT NULL
- CHECK constraints: ALTER TABLE t ADD CONSTRAINT chk_amount CHECK (amount > 0)
- DLT Expectations: @dlt.expect('valid_email', "email LIKE '%@%'")
- DLT Quarantine: @dlt.expect_or_drop, @dlt.expect_or_fail for stricter enforcement

# Section 4: Working with Lakeflow Jobs (16%)

## 4.1 Lakeflow Jobs Overview

Lakeflow Jobs (formerly Databricks Workflows) is the orchestration engine. It uses a DAG (Directed Acyclic Graph) model where tasks are nodes and dependencies are edges.

## 4.2 Task Types

| Task Type | Description |
|---|---|
| Notebook Task | Runs a Databricks notebook; pass parameters via widgets |
| SQL Query Task | Executes a saved SQL query in a SQL Warehouse |
| Dashboard Task | Refreshes a Databricks dashboard |
| Pipeline Task | Triggers a Lakeflow Spark Declarative Pipeline (DLT) |
| Python Script Task | Runs a Python .py file directly on a cluster |
| JAR Task | Executes a compiled Java/Scala JAR |
| dbt Task | Runs dbt models via dbt Core integration |
| Condition Task | Evaluates an if/else condition for branching |
| For Each Task | Loops over a list, running a task for each item |
| Run Job Task | Triggers another Lakeflow Job as a sub-job |

## 4.3 Control Flow — Retries, Branching, Looping

### Retries

- Set Max Retries per task (e.g., 3 retries on failure)
- Set Retry On Timeout for tasks with time limits
- Configure delay between retries to handle transient issues

### Branching (Condition Task)

- Use a Condition Task to evaluate a True/False expression
- Route downstream tasks based on condition outcome
- Example: if rowcount > 0 → run LoadTask, else → run SkipTask

### Looping (For Each Task)

- Accepts a JSON array as input (e.g., list of table names)
- Runs a nested task for each element in the array
- Control concurrency (how many iterations run in parallel)

## 4.4 Trigger Types

| Trigger Type | Description | When to Use |
|---|---|---|
| Scheduled | Cron expression; time-based | Daily loads, regular batch jobs |
| File Arrival | Triggers on new file in cloud storage | Event-driven ingestion pipelines |
| Table Update | Triggers when a table receives new data | Downstream dependency pipelines |
| Manual | On-demand via UI or API | Ad-hoc runs, testing |

> ⚠ **Exam Tip:** Time-based triggers run even if no new data has arrived. Data-driven triggers (File Arrival, Table Update) are more efficient for event-driven pipelines.

## 4.5 Job Parameters

- Pass parameters to notebooks using widgets: dbutils.widgets.get('param_name')
- Define job-level parameters that are passed to all tasks
- Override parameters at the task level for task-specific values
- Use {{job.id}}, {{run.id}} as dynamic system variables

# Section 5: Implementing CI/CD (10%)

## 5.1 Databricks Git Folders

Git Folders (formerly Repos) provide native Git integration inside the Databricks workspace UI.

### Key Operations

- Create new branch: from branch picker → create new branch
- Switch branch: pull dropdown → select existing branch
- Commit changes: Files panel → select changed files → write message → Commit
- Push to remote: Commit & Push or separate Push button
- Pull latest: Git operations → Pull from remote
- Create Pull Request: Link takes you to GitHub/GitLab/Azure DevOps PR UI

> **Note:** Git Folders support GitHub, GitLab, Azure DevOps, Bitbucket, and Gitea as remote providers.

## 5.2 Declarative Automation Bundles (DABs)

Formerly Databricks Asset Bundles (DABs). They package Jobs, Pipelines, and workspace assets for deployment across environments.

### Bundle Project Structure

```
my_bundle/
  databricks.yml        # Root bundle config
  resources/
    jobs/
      my_job.yml        # Job definition
    pipelines/
      my_pipeline.yml   # DLT pipeline definition
  src/
    notebook.py         # Source code
```

### Environment Targets

```
targets:
  dev:
    default: true
    workspace:
      host: https://dev.azuredatabricks.net
    variables:
      catalog: dev_catalog
  prod:
    workspace:
      host: https://prod.azuredatabricks.net
    variables:
      catalog: prod_catalog
```

### Variable References in Resources

```
resources:
  jobs:
    my_job:
      tasks:
        - task_key: etl_task
          notebook_task:
            base_parameters:
              catalog: ${var.catalog}   # References the variable
```

## 5.3 Databricks CLI — Key Commands

| Command | Purpose |
|---|---|
| `databricks bundle validate` | Validate bundle YAML syntax and structure |
| `databricks bundle deploy -t dev` | Deploy bundle to dev target |
| `databricks bundle deploy -t prod` | Deploy bundle to prod target |
| `databricks bundle run <job_name>` | Trigger a job from the bundle |
| `databricks bundle destroy -t dev` | Remove bundle resources from dev |
| `databricks jobs list` | List all jobs in the workspace |
| `databricks clusters list` | List all clusters |
| `databricks fs ls dbfs:/mnt/` | List DBFS directory contents |

# Section 6: Troubleshooting, Monitoring & Optimization (10%)

## 6.1 Monitoring Job Runs

- Run History tab shows all runs: status, duration, start time
- Compare execution times using the Duration column across runs
- Job run timeline shows task-level parallelism and bottlenecks
- Use Run Details → Task Graph to identify which task failed or is slow

## 6.2 Spark UI Interpretation

| Spark UI Tab | What to Look For |
|---|---|
| Jobs | Overall job progress; number of stages |
| Stages | Stage-level aggregates; look for high shuffle read/write |
| Tasks | Task-level details; identify skewed tasks (outlier durations) |
| Storage | Cached RDDs/DataFrames; memory usage |
| Environment | Spark config values; confirm tuning params |
| Executors | Memory used vs. max; GC time; disk spill |
| SQL / Dataframe | Query plans; identify expensive joins or sorts |

## 6.3 Common Performance Bottlenecks

### Data Skew

- Symptom: Most tasks complete quickly; a few tasks run 10–100× longer
- Cause: Uneven data distribution across partitions (hot keys)
- Fix: Salting — add a random prefix to skewed keys, then aggregate twice
- Fix: Use AQE (Adaptive Query Execution) — skew join optimization is automatic

### Excessive Shuffling

- Symptom: High shuffle read/write bytes in Spark UI; many stages
- Cause: Wide transformations (groupBy, join, distinct) across many partitions
- Fix: Reduce spark.sql.shuffle.partitions for small data
- Fix: Broadcast small tables to avoid shuffle in joins
- Fix: Partition data by join/filter keys to minimize data movement

### Disk Spilling

- Symptom: 'Spill (disk)' column in Spark UI shows non-zero values
- Cause: Data doesn't fit in executor memory during aggregation or sort
- Fix: Increase spark.executor.memory or reduce partition size
- Fix: Increase number of shuffle partitions to reduce per-partition data

## 6.4 Liquid Clustering & Predictive Optimization

### Liquid Clustering

- Replaces OPTIMIZE + Z-Order with automatic, incremental clustering
- Defined with: CLUSTER BY (col1, col2) at table creation or via ALTER TABLE
- No need to run OPTIMIZE manually — clustering is incremental on ingestion
- Supports changing cluster keys without rewriting the entire table
- Best for: tables with high write rates and multi-column query patterns

### Predictive Optimization

- Automatically runs OPTIMIZE and VACUUM on tables based on usage patterns
- Enabled at the Unity Catalog metastore level
- No manual OPTIMIZE / VACUUM scheduling required

> **Note:** Predictive Optimization is a Unity Catalog feature. It only applies to tables managed by Unity Catalog.

## 6.5 Common Cluster Issues

| Issue | Diagnosis / Fix |
|---|---|
| Cluster startup failure | Check Event Log; often a misconfigured init script or quota limit |
| Library conflicts | Use Cluster-scoped vs. Notebook-scoped libraries carefully; avoid version mismatches |
| OOM (Out of Memory) | Increase executor memory; check for collect() / broadcast of large data |
| Long GC pauses | Too much object creation; reduce shuffle data; increase off-heap memory |
| Autoscaling not triggered | Check autoscale min/max settings; verify load metric thresholds |

# Section 7: Governance and Security (15%)

## 7.1 Managed vs. External Tables

| | Managed Table | External Table |
|---|---|---|
| Data location | Managed by Unity Catalog (default location) | User-specified external location |
| DROP TABLE effect | Deletes both metadata AND data files | Deletes only metadata; data remains |
| CLONE support | Yes | Yes |
| Governance | Full Unity Catalog governance | Full Unity Catalog governance |
| When to use | Default for most tables | When data must stay in existing location |

### Table Operations

```
-- Create managed table
CREATE TABLE catalog.schema.my_table (id INT, name STRING) USING DELTA;

-- Create external table
CREATE TABLE catalog.schema.ext_table USING DELTA
LOCATION 'abfss://container@acct.dfs.core.windows.net/path/';

-- Convert external to managed (CLONE approach)
CREATE TABLE catalog.schema.managed_table CLONE catalog.schema.ext_table;
```

## 7.2 Access Control — GRANT, REVOKE, DENY

### Privilege Hierarchy

- Metastore → Catalog → Schema → Table/View
- Privileges granted at higher levels cascade down automatically
- DENY at any level overrides GRANT at any level

### Common Privilege SQL Examples

```
-- Grant SELECT on a table to a group
GRANT SELECT ON TABLE catalog.schema.orders TO GROUP data_analysts;

-- Grant ALL PRIVILEGES on schema
GRANT ALL PRIVILEGES ON SCHEMA catalog.my_schema TO USER parth@company.com;

-- Revoke a privilege
REVOKE SELECT ON TABLE catalog.schema.orders FROM GROUP data_analysts;

-- Deny privilege (overrides any GRANT)
```

```
DENY SELECT ON TABLE catalog.schema.sensitive TO GROUP contractors;

-- Show grants on an object
SHOW GRANTS ON TABLE catalog.schema.orders;
```

| Privilege | Applies To |
|---|---|
| SELECT | Tables, Views, Materialized Views |
| MODIFY | Tables (INSERT, UPDATE, DELETE, MERGE) |
| CREATE TABLE | Schemas |
| CREATE SCHEMA | Catalogs |
| USE CATALOG | Catalogs (required to access objects within) |
| USE SCHEMA | Schemas (required to access objects within) |
| ALL PRIVILEGES | Any securable object |

## 7.3 Column-Level Security

### Column Masking

- Hides or transforms column values based on the user's group membership
- Applied via a SQL masking function attached to the column

```
-- Create masking function
CREATE FUNCTION catalog.schema.mask_ssn(ssn STRING)
RETURNS STRING
RETURN CASE WHEN is_member('hr_team') THEN ssn ELSE 'XXX-XX-' || RIGHT(ssn, 4) END;

-- Apply mask to column
ALTER TABLE catalog.schema.employees
ALTER COLUMN ssn SET MASK catalog.schema.mask_ssn;
```

## 7.4 Row-Level Security

### Row Filters

- Filters rows returned based on the current user's group membership
- Defined as a SQL function returning a BOOLEAN; applied as a table-level filter

```
-- Create row filter function
CREATE FUNCTION catalog.schema.region_filter(region STRING)
RETURNS BOOLEAN
RETURN is_member('global_users') OR region = session_context('user_region');

-- Apply row filter to table
ALTER TABLE catalog.schema.sales
SET ROW FILTER catalog.schema.region_filter ON (region);
```

⚠ **Exam Tip:** Column masking and row filters are enforced at query time — they apply to ALL access paths including notebooks, SQL, and BI tools.

## 7.5 ABAC — Attribute-Based Access Control

- ABAC policies centrally control row-level filtering and column masking
- Policies are defined using Unity Catalog's policy framework
- ABAC replaces the need to build custom masking functions per table
- Attributes can be user attributes, group memberships, or data classifications

> **Note:** ABAC policies are managed at the Unity Catalog level and automatically propagate to all governed tables.

# Quick Reference Cheatsheet

## Must-Know Commands

```
-- COPY INTO (incremental load)
COPY INTO catalog.schema.table FROM 'cloud://path/' FILEFORMAT = PARQUET;

-- Auto Loader (streaming)
spark.readStream.format('cloudFiles').option('cloudFiles.format','json').load('path')

-- Broadcast join
from pyspark.sql.functions import broadcast
df1.join(broadcast(df2), 'key_col')

-- Window deduplication
w = Window.partitionBy('id').orderBy(col('ts').desc())
df.withColumn('rn', row_number().over(w)).filter(col('rn')==1).drop('rn')

-- Grant access
GRANT SELECT ON TABLE cat.sch.tbl TO GROUP analysts;

-- Apply column mask
ALTER TABLE cat.sch.emp ALTER COLUMN ssn SET MASK cat.sch.mask_fn;

-- Liquid Clustering
ALTER TABLE cat.sch.tbl CLUSTER BY (col1, col2);

-- DABs deploy
databricks bundle deploy -t prod
```

## Key Numbers to Remember

| Fact | Value |
|---|---|
| Default shuffle partitions | 200 |
| Broadcast join threshold default | 10 MB |
| approx_count_distinct error rate | ~2% |
| Highest weight exam sections | Transformation (22%) & Ingestion (21%) |
| Unity Catalog hierarchy levels | 4: Metastore → Catalog → Schema → Object |
| DENY overrides GRANT | Always — at any hierarchy level |
| DROP managed table effect | Deletes both data and metadata |
| DROP external table effect | Deletes only metadata; data files remain |